

CIMENTA

Bitácora del proyecto

Inteligencia inmobiliaria cuantitativa · Ciudad de México · 2026

Registro cronológico de todo lo que hacemos en el proyecto. La entrada más reciente va arriba.

Convención de cada entrada: fecha y responsable · qué se hizo · decisiones / notas · pendientes / siguiente paso.

2026-06-13 — Inmuebles24 vía CloakBrowser + geocoding por colonia

Decisión

Se agotaron los portales fáciles con coordenadas (CyT + Lamudi). Pincali no trae coords (es EasyBroker por debajo); iCasas las tiene pero su HTML es frágil. Se decidió ir por los 3 grandes (mayor inventario) con la técnica **CloakBrowser** del repo FotMob — Nivel B, decisión consciente del dueño pese al “no evadir” del catálogo. Se empezó por **Inmuebles24**.

Qué se hizo

- `src/ingesta/browser.py`: sesión CloakBrowser (warmup resuelve Cloudflare una vez, reutiliza la sesión). Confirmado: pasa el Cloudflare de Inmuebles24 (título real “4,849 Departamentos...”).
- `src/ingesta/inmuebles24.py`: scraper que parsea las tarjetas **posting** con **BeautifulSoup** (id, precio, dirección, colonia/alcaldía, m²/rec/baños/estac, URL, tipo PROPERTY/DEVELOPMENT); pagina con `-pagina-N.html`; dedup por id; JSONL + CSV; reanudable.
- `src/ingesta/geocode.py`: **geocodificación por colonia** vía OSM/Nominatim (1 req/s, User-Agent identificable, viewport ZMVM, caché en `data/geocode_cache.json`). Reutilizable para cualquier portal sin coords.
- Dependencias: `cloakbrowser`, `beautifulsoup4`.

Prueba (pipeline completo)

- Scrape BJ depts venta (2 pág): 51 postings, todos con precio/colonia/m². Paginación OK.
- Geocode: 27 colonias únicas → **50/51 filas con coordenadas** a nivel colonia, cacheadas.

Caveats

- Inmuebles24 **no expone coordenadas** → posición a nivel colonia (aproximada), no exacta.
- Tarjetas DEVELOPMENT (preventa) traen precios/m² en rango; se marcan con `card_type` para poder filtrarlas.
- El scrape es por navegador → lento (horas para CDMX+ZMVM completo). Reanudable.

Pendientes / siguiente paso

- ☐ Correr Inmuebles24 completo + geocode (comandos abajo).
- ☐ Replicar a Propiedades.com y Vivanuncios (mismo patrón CloakBrowser).
- ☐ Unificar CSVs de portales para que el visor muestre todos juntos.

2026-06-13 — Corrida completa CyT (tqdm) + scraper Lamudi

Casas y Terrenos — listo para corrida completa

- Se añadió barra de progreso `tqdm` al scraper (combo actual, %, ETA, acumulado). Dependencia `tqdm` agregada.
- Corrida completa = 434 combos (16 alcaldías CDMX + 15 municipios ZMVM × 7 tipos × 2 operaciones). Reanudable; continúa desde el checkpoint (Benito Juárez ya estaba hecho).
- Comando: `python -m src.ingesta.casasyterrenos`.

Lamudi — segundo portal Nivel A (scraper nuevo)

- `robots.txt` permite los listados (bloquea forms/prefetch y URLs con `search-filter/action/version`).
- Server-rendered con **JSON-LD**: los anuncios están en `@graph[0].mainEntity[0].itemListElement[*].itemListElement` — cada uno schema.org con `name`, `@type`, `numberOfBedrooms`, `numberOfBathroomsTotal`, `floorSize` (m²), `geo` (lat/lng), `address`, `offers` (precio) y `url`.
- `src/ingesta/lamudi.py`: URL `/ {estado} / {operacion} / ?page=N` (30/pág, `for-sale/for-rent`); pagina hasta vaciar o `max_pages`; dedup por id de URL; JSONL + CSV; reanudable.
- Se corrigió **mojibake** (Lamudi no declara charset → se fuerza UTF-8).
- **Caveat**: Lamudi **no expone fecha de publicación** en el listado → se toman anuncios **activos** (sin filtro de 3 meses). Para el mapa es justo lo deseado (inventario vigente).
- Probado: distrito-federal/for-sale → 30 listings/pág con todos los campos y acentos correctos.

Pendientes / siguiente paso

- ☐ Correr la completa de Lamudi (`python -m src.ingesta.lamudi`).
- ☐ Unificar CSVs de portales en un esquema común para el visor.
- ☐ Siguientes Nivel A: Pincali, iCasas, EasyBroker (API).

2026-06-13 — Visor Streamlit + mapa OpenStreetMap

Qué se hizo

- Dashboard minimalista `app/streamlit_app.py` para explorar el CSV scrapeado (855 propiedades de Benito Juárez).

- **Mapa OpenStreetMap** con `folium` + `streamlit-folium`: marcadores en clúster, color por operación (venta/renta), popups con precio, m², recámaras, baños, colonia y link al anuncio. Base seleccionable (CartoDB Positron minimalista / OpenStreetMap).
- Filtros (operación, tipo, municipio, recámaras), KPIs (conteo, venta/renta, precio mediano de venta, m² mediano) y tabla de detalle con links.
- Estilo con la paleta CIMENTA (Inter + Space Grotesk), menú/footer ocultos.
- Dependencias: `streamlit`, `pandas`, `folium`, `streamlit-folium`.
- Probado con `AppTest` (corre sin excepción; se corrigió un bug de NaN en popups). Corre en `localhost:8501`.

Cómo correrlo

- `conda activate cimenta_env && streamlit run app/streamlit_app.py`

2026-06-13 — Fotos de propiedades → Google Drive + borrado local (M1)

Qué se hizo

- Las propiedades de Casas y Terrenos traen `images` = lista de URLs directas a S3 (`cyt-media.s3...`). Descarga directa, sin auth.
- Se reutilizó el patrón de Drive del repo *Prueba_fotmob* (OAuth Desktop, jerarquía de carpetas, resumable, log de subidos), **añadiendo la pieza que faltaba: borrar local tras subir**.
- Nuevo `src/ingesta/fotos_a_drive.py` — flujo **rolling**: por propiedad → descarga fotos → ZIP `{id}.zip` → sube a Drive → borra local. Así en disco nunca hay más que una propiedad a la vez.
- Destino: Mi unidad / CIMENTA / `{portal}` / fotos / `{id}.zip`.
- Credenciales (`credentials.json`, `token.json`) copiadas del proyecto FotMob y **gitignored** (nunca se versionan).
- Dependencias: `google-api-python-client`, `google-auth-oauthlib`, `google-auth-httpplib2`.

Prueba

- Cuenta Drive: `danielomar.becerrilolguin@gmail.com` — 163 GB / 5 TB usados (espacio de sobra).
- 3 propiedades de prueba → 3 ZIPs en Drive (5.2 / 1.8 / 0.3 MB), temporal local borrado, **resume** verificado (re-correr salta los 3).

Pendientes / siguiente paso

- ☐ Correr la subida de fotos para todas las propiedades scrapeadas.
- ☐ Encadenar scraping → fotos como pipeline (o correrlos por separado).

2026-06-13 — Recon de los 10 portales + PoC scraping (M1)

Recon técnico de los 10 (clasificación Nivel A/B)

Se probó cada portal (status, anti-bot, forma de entrega de datos). Resultado:

#	Portal	Anti-bot / entrega	Nivel
1	Inmuebles24	Cloudflare challenge (403)	B
2	Propiedades.com	Cloudflare challenge	B
3	Vivanuncios	Cloudflare “Just a moment” (403)	B
4	Lamudi	accesible (Jetty), HTML	A
5	Mercado Libre	accesible; protección profunda/API	A/B
6	Pincali	accesible (Rails), JSON-LD	A
7	Trovit	401 (agregador, requiere headers)	—
8	EasyBroker	accesible; API oficial documentada	A+
9	iCasas	accesible (Apache), HTML	A
10	Casas y Terrenos	accesible; <code>__NEXT_DATA__</code>	A

Los 3 más grandes están tras Cloudflare → Nivel B (su propio catálogo dice no evadir sin acuerdo).
Los 7 restantes son accesibles con HTTP normal.

PoC — Casas y Terrenos (Nivel A)

- `robots.txt` permite los listados para `User-Agent: *` (solo bloquea `/api/`, `/beta/` y bots SEO). Sin anti-bot.
- Next.js: los listados vienen en `__NEXT_DATA__` → `props.pageProps.initialState.propertyData.propertyData`. URL `/{{estado}}/{{municipio}}/{{tipo}}/{{operacion}}?page=N` (240/pág, 1-indexed); total en `estimatedTotalHits`.
- Se construyó `src/ingesta/casasyterrenos.py` (config-driven desde `config.yml`): itera alcaldías CDMX + municipios ZMVM × tipos × operaciones, pagina, filtra por `lastUpdate` (3 meses), deduplica por `id`, guarda JSONL (raw) + CSV (plano). Reanudable.
- Dependencias de la etapa: `requests`, `PyYAML` (instaladas en `cimenta_env`).
- Prueba (Benito Juárez, 14 combos, ~45 s): 855 propiedades únicas ≤3 meses, dedup perfecto (855 CSV = 855 JSONL = 855 ids).

Caveats de calidad de datos (anotados)

- La búsqueda por alcaldía es **geo-radial / con match de texto**: un seed devuelve propiedades de zonas vecinas. Se mitiga con dedup por `id` y la unión de todas las alcaldías.
- El campo `municipality` lo captura el broker y **a veces es incorrecto** (ej. un anuncio con coords en CDMX etiquetado “Morelia”). La señal confiable de ubicación es `_geo` (lat/lng).
- `lastUpdate` es **última actualización**, no fecha de publicación pura (el listado no la expone). Se usa como proxy de “reciente”.

Pendientes / siguiente paso

- ☐ Lanzar la corrida completa CDMX + ZMVM de Casas y Terrenos (434 combos).
- ☐ Replicar el patrón a los otros portales Nivel A (Lamudi, Pincali, iCasas, EasyBroker, Casas y Terrenos).

- ☐ Decidir qué hacer con los 3 de Cloudflare (CloakBrowser vs. acuerdo vs. excluir).

2026-06-12 — Etapa 1: portales objetivo (M1 Ingesta)

Qué se hizo

- Investigación de los principales portales para comprar inmuebles en CDMX y la ZMVM, cruzando rankings de tráfico de **Similarweb** (top 5, may-2026) y **Semrush** (abr-2026, visitas/mes).
- Se excluyeron sitios con poco inventario CDMX o no-portal (craigslist, zillow, idealista, foros).
- Se creó `config.yml` con la lista `portales` (objetos `{nombre, sitio}`).

Top 10 portales seleccionados

#	Portal	Sitio
1	Inmuebles24	https://www.inmuebles24.com
2	Propiedades.com	https://propiedades.com
3	Vivanuncios	https://www.vivanuncios.com.mx
4	Lamudi	https://www.lamudi.com.mx
5	Mercado Libre Inmuebles	https://inmuebles.mercadolibre.com.mx
6	Pincali	https://www.pincali.com
7	Trovit Casas	https://casas.trovit.com.mx
8	EasyBroker	https://www.easybroker.com
9	iCasas	https://www.icasas.mx
10	Casas y Terrenos	https://www.casasyterrenos.com

Decisiones / notas

- Orden por tráfico nacional; todos tienen inventario en CDMX/ZMVM.
- Trovit es agregador (útil para probar deduplicación en M1); EasyBroker es backend tipo MLS con API. Alternos a futuro: Metroscubicos, Century21, RE/MAX MX.

Pendientes / siguiente paso

- ☐ Definir la estructura de M1 (Ingesta): cómo recolectar de cada portal.
- ☐ Revisar términos de uso / `robots.txt` de cada sitio antes de scrapear.

2026-06-12 — Arranque del proyecto (Etapa 0: setup)

Qué se hizo

- Estructura base del repositorio: `app/`, `data/`, `notebooks/`, `test/`, `src/`, `docs/`, más `README.md`, `requirements.txt`, `.gitignore` y esta bitácora.
- Carpetas vacías versionadas con `.gitkeep`.
- Entorno de Anaconda `cimenta_env` con **Python 3.12** (vacío, sin dependencias todavía).
- Repositorio Git con tres ramas: `master`, `dev`, `feature/dev`.

- Remoto configurado a https://github.com/danielomar14/cimenta_repo.git y las tres ramas subidas.

Decisiones / notas

- Flujo de trabajo: desarrollo en `feature/dev` → pruebas en `dev` → estable en `master`.
- El entorno se mantiene vacío a propósito; las dependencias se agregarán etapa por etapa.
- Datos (`data/`) excluidos del control de versiones vía `.gitignore`.
- La bitácora se lleva en LaTeX (`BITACORA.tex`) y se renderiza a PDF (`BITACORA.pdf`) con `tectonic`.

Bloqueo detectado — Catálogo de Etapas ilegible

- El archivo `docs/context/CIMENTA-Catalogo-de-Etapas.pdf` (236 págs.) está **corrupto**: todos los bytes $\geq 0x80$ de sus streams comprimidos fueron sustituidos por el carácter de reemplazo Unicode U+FFFD (`ef bf bd`) — 823,069 ocurrencias. Es resultado de un round-trip UTF-8 que destruyó la data binaria. **No es recuperable**: ni texto ni render.
- Los demás PDFs de `docs/context/` están sanos: Plan-Maestro-v2 (97p), Plan-Maestro (43p), Bitácora-del-Jurado (120p), Propuesta-Inicial (12p), Riesgos-y-Debilidades (17p).
- **Acción requerida**: reexportar / volver a subir una copia limpia del Catálogo de Etapas para poder avanzar etapa por etapa.

Pendientes / siguiente paso

- ☐ Obtener copia limpia de `CIMENTA-Catalogo-de-Etapas.pdf`.
- ☐ Leer el catálogo y definir la Etapa 1.